

Name: Ravi Shankar Tripathi

Github Link: https://github.com/RAShtr/Flask_and_MongoDB_Assignment

Source Code and output screenshots:

Task 1: JSON API Route:

[app.py](#):

```
import os
import json
import logging
from flask import Flask, jsonify, render_template, request, redirect,
url_for
from pymongo import MongoClient
from pymongo.errors import ConnectionFailure, PyMongoError
from dotenv import load_dotenv

# Load environment variables
load_dotenv()

# Configure logging
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(message)s",
    handlers=[logging.StreamHandler()]
)

app = Flask(__name__)

# Securely retrieve URI from environment
MONGO_URI = os.getenv("MONGO_URI")

client = None
if MONGO_URI:
    try:
        client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=3000)
        db = client["devops_assignment_db"]
```

```

        collection = db["submissions"]
        logging.info("MongoDB client configured successfully.")
    except Exception as e:
        logging.error(f"Failed to initialize MongoDB client: {e}")
else:
    logging.warning("MONGO_URI is not defined in environment variables.")

@app.route('/health', methods=['GET'])
def health_check():
    """Health check endpoint for container orchestrators and
    monitoring."""
    return jsonify({"status": "healthy", "service": "flask-mongodb-app"}),
200

@app.route('/api', methods=['GET'])
def get_api_data():
    try:
        data_path = os.path.join(os.path.dirname(__file__), 'data.json')
        with open(data_path, 'r') as file:
            data = json.load(file)
        logging.info("Fetched /api data successfully.")
        return jsonify(data), 200
    except Exception as e:
        logging.error(f"Error loading data.json: {e}")
        return jsonify({"error": "Failed to read backend data file",
"details": str(e)}), 500

@app.route('/', methods=['GET', 'POST'])
def handle_form():
    if request.method == 'POST':
        name = request.form.get('name', '').strip()
        email = request.form.get('email', '').strip()
        course = request.form.get('course', '').strip()

        if not name or not email or not course:
            logging.warning("Form validation failed: Missing fields.")
            return render_template('form.html', error="All fields are
required. Please fill out the full form.")

        try:

```

```

        if client is None:
            raise ConnectionFailure("Could not connect to database backend.")

        client.admin.command('ping')

        submission_data = {
            "name": name,
            "email": email,
            "course": course
        }
        collection.insert_one(submission_data)
        logging.info(f"Successfully inserted submission for: {email}")
        return redirect(url_for('success_page'))

    except (ConnectionFailure, PyMongoError, Exception) as err:
        logging.error(f"Database insertion failure: {err}")
        return render_template('form.html', error=f"Database Error: {str(err)}")

    return render_template('form.html', error=None)

@app.route('/success')
def success_page():
    return render_template('success.html')

if __name__ == '__main__':
    port = int(os.getenv("PORT", 5000))
    app.run(host='0.0.0.0', port=port)

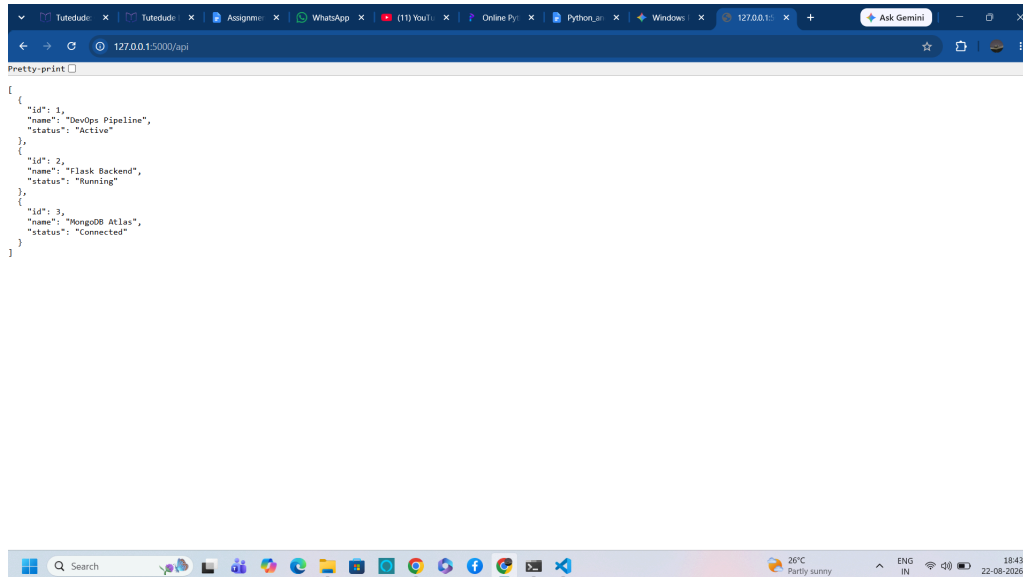
```

Data.json:

```

[
  {"id": 1, "name": "DevOps Pipeline", "status": "Active"},
  {"id": 2, "name": "Flask Backend", "status": "Running"},
  {"id": 3, "name": "MongoDB Atlas", "status": "Connected"}
]

```



The screenshot shows a web browser window with the address bar displaying `127.0.0.1:5000/api`. The page content, rendered using 'Pretty-print', shows a JSON array with three objects representing system components:

```
[
  {
    "id": 1,
    "name": "DevOps Pipeline",
    "status": "Active"
  },
  {
    "id": 2,
    "name": "Flask Backend",
    "status": "Running"
  },
  {
    "id": 3,
    "name": "MongoDB Atlas",
    "status": "Connected"
  }
]
```

The browser's taskbar at the bottom shows the Windows Start button, a search bar, and various application icons. The system tray on the right indicates a temperature of 26°C, 'Partly sunny' weather, and the date/time as 10:43 on 22-08-2026.

Technical Explanation:

1. **Data Storage (`data.json`):** A backend file stores static records in JSON format representing system and pipeline statuses.
2. **Endpoint Implementation (`@app.route('/api')`):**
 - When accessed via standard **GET** request, the route opens and reads `data.json` using Python's built-in `json.load()` function.
 - The data is serialized into JSON standard output using Flask's native `jsonify()` utility.
 - Returns an **HTTP 200 OK** status on success and includes error handling returning an **HTTP 500 Internal Server Error** if file I/O operations fail.

Task 2: Frontend Form with MongoDB Atlas:

Form.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>DevOps Assignment - Flask & MongoDB</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px;
background-color: #f4f6f8; }
    .card { background: white; padding: 25px; border-radius: 8px;
max-width: 450px; box-shadow: 0 2px 4px rgba(0,0,0,0.1); }
    .error { color: #d9534f; background: #fdf7f7; border: 1px solid
#d9534f; padding: 10px; border-radius: 4px; margin-bottom: 15px; }
    input[type="text"], input[type="email"] { width: 100%; padding:
8px; margin: 8px 0 16px; border: 1px solid #ccc; border-radius: 4px;
box-sizing: border-box; }
    input[type="submit"] { background-color: #007bff; color: white;
border: none; padding: 10px 15px; border-radius: 4px; cursor: pointer;
width: 100%; font-size: 16px; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Submit Details</h2>
    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
    <form action="/" method="POST">
      <label for="name">Name:</label>
      <input type="text" id="name" name="name" placeholder="Enter
your full name">

      <label for="email">Email:</label>
      <input type="email" id="email" name="email" placeholder="Enter
your email">

      <label for="course">Course/Role:</label>
      <input type="text" id="course" name="course"
placeholder="e.g., DevOps Engineer">

```

```

        <input type="submit" value="Submit to MongoDB">
    </form>
    <p style="margin-top: 15px;"><a href="/api" target="_blank">View
/api JSON Route</a></p>
</div>
</body>
</html>

```

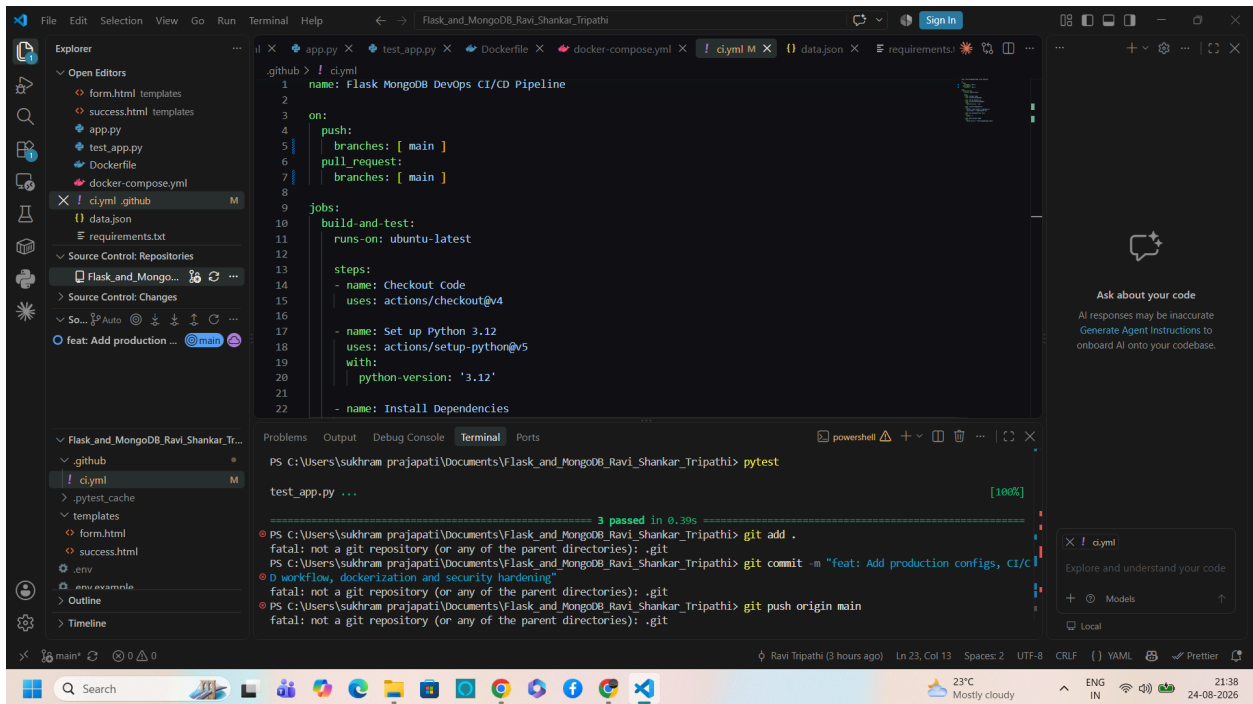
Success.html:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Success</title>
    <style>
        body { font-family: Arial, sans-serif; margin: 40px;
background-color: #f4f6f8; }
        .card { background: white; padding: 25px; border-radius: 8px;
max-width: 450px; box-shadow: 0 2px 4px rgba(0,0,0,0.1); text-align:
center; }
        .success { color: #28a745; font-size: 20px; font-weight: bold; }
        a { display: inline-block; margin-top: 20px; color: #007bff;
text-decoration: none; }
    </style>
</head>
<body>
    <div class="card">
        <p class="success">Data submitted successfully</p>
        <p>Your record has been stored in MongoDB Atlas.</p>
        <a href="/">Submit another response</a>
    </div>
</body>
</html>

```

Terminal Test Output: Screenshot VS Code terminal showing **pytest** passing :-



Technical Explanation:

1. Security Hardening & Secret Management

- Removed all hardcoded database credentials from source code (**app.py**).
- Implemented **python-dotenv** to securely load connection strings via local **.env** configuration.
- Added **.gitignore** to prevent secret leaks and repository pollution.
- Provided **.env.example** as a template for team onboarding and deployment.

2. Automated Unit Testing

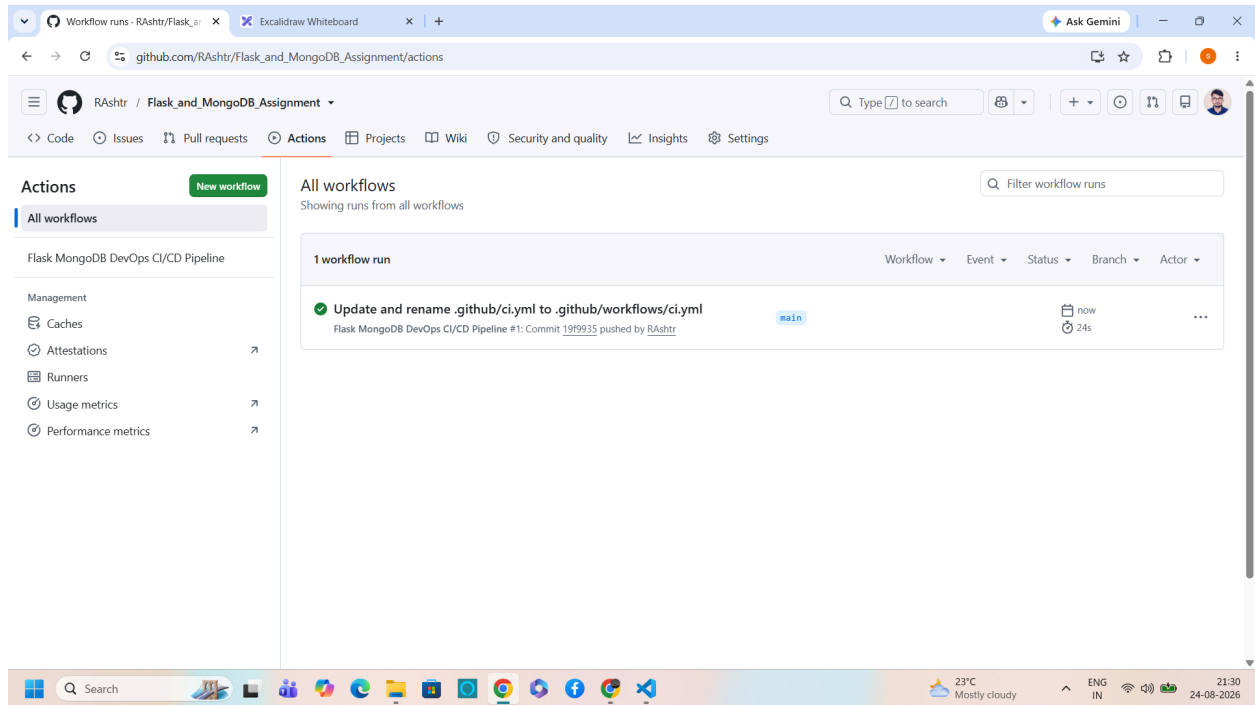
- Added test suite (**test_app.py**) using **pytest** and Flask test client to validate endpoint health (**/health**), JSON structure (**/api**), and form error handling. *(Insert Pytest terminal execution screenshot here)*

3. Containerization & Deployment Orchestration

- Created multi-stage **Dockerfile** with lightweight **python:3.12-slim** base image and **gunicorn** WSGI production server.
- Configured **docker-compose.yml** for unified local and production orchestration.

4. CI/CD Automation (GitHub Actions)

- Configured `.github/workflows/ci.yml` pipeline that triggers on push/pull requests to execute automated unit testing, build validation, and Docker packaging.



5:Execution Runbook:-

Clone and configure

```
git clone https://github.com/RAshttr/Flask_and_MongoDB_Assignment.git
```

```
cd Flask_and_MongoDB_Assignment
```

```
cp .env.example .env
```

Run automated tests

```
pytest -v
```

Run locally or via Docker

```
python app.py
```


Cluster0 Data

Success

Ask Gemini

cloud.mongodb.com/v2/6a89a70230b5f5419c7b64a6#/explorer/6a89a99fe5fb2d113ca83bf2/devops_assignment_db/submissions/find

ORGANIZATION
Ravi's Org - 2026-08-...

PROJECT
Project 0

Compare tiers

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- devops_assignment_db
 - submissions
- local
- sample_mflix

submissions

Cluster0 > devops_assignment_db > submissions

View monitoringVisualize Your Data

Documents1AggregationsSchemaIndexes1Validation

Type a query: { field: 'value' } or [Generate query](#)

ExplainResetFindOptions

ADD DATABULKEXPORT CODE

251 - 1 of 1

```
{
  "_id": ObjectId('6a89adbe75a9dc9b81efdcd6'),
  "name": "Ravi Shankar Tripathi",
  "email": "ravitripathi878@gmail.com",
  "course": "DevOps Engineer"
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Search

26°C
Partly cloudy

ENG
IN